TripSync : Plan, Pack and Go

Software Requirements Specification

Prepared by

Kaspia Khan Onti -23101145
Nabila Haque-23301018
Lamia Hai Meghla-23301073
Mehjabin Iqbal Efa-23101268

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the requirements for **TripSync: Plan, Pack, and Go**, a web-based travel planning application built using the **MERN stack** (MongoDB, Express.js, React.js, Node.js).

The application aims to provide travelers with a unified platform to plan, organize, and track all aspects of their trips. It allows users to manage itineraries, packing checklists, budgets, travel documents, and group collaborations, while also providing real-time information such as weather, currency exchange rates, and flight statuses.

TripSync ensures efficient, secure, and interactive travel management, helping users plan smart, pack efficiently, and travel stress-free.

1.2 Type of Project

TripSync is a **web-based travel management application** with a focus on **planning, tracking, and collaboration**. The system is a feature-rich MERN stack application designed to handle CRUD operations for trips, itineraries, expenses, and user data.

It integrates with **external APIs** (weather, maps and API endpoints

Database: MongoDB for persistent storage of trips, users, and travel data , currency, flights) to provide real-time travel information, while maintaining a responsive and user-friendly interface. The project is full-stack, including:

Frontend: React.js for dynamic and responsive UI

Backend: Node.js + Express.js for server-side logic an

1.3 Target Users

The primary target users of TripSync are:

1. **Individual Travelers:**
    - Plan personal or family trips with a complete itinerary, budget, and document management.
    - Maintain travel journals and track past trips.
2. **Group Travelers:**
    - Collaborate on shared trips with friends or colleagues.
    - Manage group expenses, shared itineraries, and travel activities.
3. **Frequent Travelers:**
    - Keep all travel-related information centralized, including flight details, documents, and favorite locations.
    - Benefit from automated itinerary suggestions and real-time updates.

The system assumes users have basic computer literacy and access to a modern web browser with internet connectivity.

2 . Functional Requirements

2.1 Trip Planning & Management

- **FR-1:** Users can create, view, edit, and delete trips. (Didn't got the codes)

- **FR-2:** Dashboard shows all upcoming and past trips. (Nabila)

- **FR-3:** Users can duplicate existing trips to create new ones quickly with editable details, allowing efficient reuse of past itineraries and preferences. (Nabila)

2.2 Budgeting & Expense Management

- **FR-4:** Users can set total trip budget and log categorized expenses. (Nabila)
- **FR-5:** The system provides AI-based budget recommendations by analyzing destination cost indexes, trip duration, and previous spending patterns.(Kaspia)

2.3 Location & Map Features

- FR-6: Users can search and save favorite places using Google Maps API and Users can mark dream destinations with custom pins.(Lamia)

- FR-7: Users can explore nearby attractions, restaurants, and transportation hubs around each itinerary stop using integrated location-based search and category filters.

2.4 Information & Tracking

- FR-8: Display real-time weather for destinations.

- FR-9: Display currency conversion using live API data.

- FR-10: Show real-time flight information (status, gate, delay).

2.5 Collaboration & Sharing

- FR-11: Users can invite collaborators to join a shared trip. (Lamia)

- **FR-12:** Users can generate shareable summaries or countdowns for social media. (Kaspia)

2.6 Journaling & Reviews

- FR-13: Users can create trip journals with text and photos. (Nabila)

- FR-14: Users can post reviews or ratings for activities. (Lamia)

2.7 Document Management

- FR-15: Users can securely upload and store travel documents (PDF, image).

2.8 User Profile

- FR-16: Users can set personalized travel goals (e.g., number of countries to visit per year or savings target for trips) and track progress through visual dashboards. (Nabila)

2.9 Utility Features

- FR-17: Auto-display of destination time zones. (Kaspia)

- FR-18: Quick itinerary generator based on basic trip input. (Lamia)

- FR-19: Users can export complete trip reports (itinerary, budget summary, documents) as downloadable PDFs or shareable links for backup or offline reference.
- FR-20: The system includes a multilingual interface, allowing users to switch between languages (e.g., English, Bangla, French) for global accessibility. (Kaspia)

3. Non-Functional Requirements

3.1 Performance

- NFR-1: Dashboard should load within 3 seconds on a standard connection.

- NFR-2: Average API response time under 500ms.

3.2 Security

- NFR-3: Use HTTPS for all data transmission.

- NFR-4: Use JWT for authentication and hashed passwords .k

- NFR-5: Secure file uploads using safe directory storage and validation.

3.3 Usability

- NFR-6: Intuitive and consistent UI with responsive design for all screen sizes.

- NFR-7: Clear navigation structure with minimal learning curve.

3.4 Reliability & Availability

- NFR-8: Maintain 99% uptime during normal operation.
- NFR-9: Database backup daily; auto-recovery supported.

3.5 Maintainability & Scalability

- NFR-10: Modular codebase (React components, Express routes).
- NFR-11: Scalable architecture to handle 1000+ concurrent users.
- NFR-12: Well-documented APIs and error handling for future maintenance.

3.6 Compatibility

- NFR-13: Compatible with Chrome, Firefox, Edge, and Safari.
- NFR-14: Works seamlessly across desktops, tablets, and mobiles.

3.7 Data Integrity & Privacy

- NFR-15: All user inputs validated before storage.
- NFR-16: No third-party sharing of personal or location data.
- NFR-17: Complies with GDPR-style privacy standards.